



AGH

AGH UNIVERSITY OF SCIENCE
AND TECHNOLOGY

PROGRAMOWANIE GRAFIKI 3D

Standard X3D

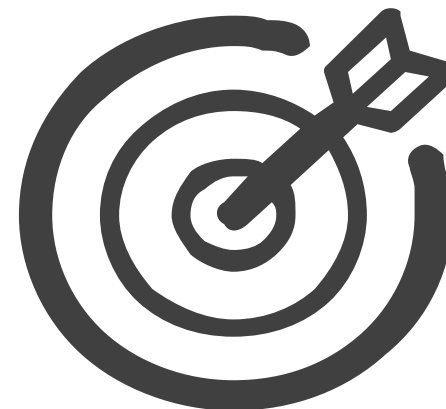
dr hab. inż. Marcin Hojny, prof. AGH

Faculty of Metals Engineering and Industrial Computer Science
Department of Applied Computer Science and Modelling

CEL ZAJĘĆ?



ANIMACJE;



CZAS ANIMACJI - WĘZEL TIMESENSOR

Węzeł **TimeSensor** należy do grupy węzłów czujnikowych odpowiedzialnych za interakcję między użytkownikiem a światem wirtualnym, zajmując się **kontrolowaniem czasu** w jakim te wszystkie operacje mają się odbywać. Interfejs węzła **TimeSensor** jest następujący:

```
Nazwa_węzła {  
  exposedField SFTime ...  
  ...  
  eventOut      SFTime  ...  
  eventIn       SFFloat  ...  
  field          SFVec3f  
}
```

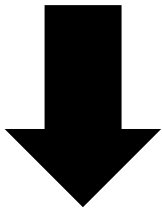
eventOut – wystąpiło zdarzenie, powiadamiamy inne węzły.

eventIn – wejście które może zmienić stan węzła.

Field - pole danych przechowujące wartość : nie wysyła zdarzeń i nie reaguje na bezpośrednie wejścia.

ELEMENTY SKŁADOWE ŚWIATA X3D

exposedField SFVec3f translation 0 0 0



field SFVec3f translation 0 0 0

eventIn SFVec3f **set_**translation

eventOut SFVec3f translation_**changed**

Słowo kluczowe **exposedField** oznacza pole, które można zarówno odczytywać, jak i modyfikować w czasie działania sceny, oraz które automatycznie tworzy dwa zdarzenia:

1. Jedno do ustawiania wartości (wejście).
2. Drugie do wysyłania zmian (wyjście).

CZAS ANIMACJI - WĘZEL TIMESENSOR

Węzeł **TimeSensor** należy do grupy węzłów czujnikowych odpowiedzialnych za interakcję między użytkownikiem a światem wirtualnym, zajmując się **kontrolowaniem czasu** w jakim te wszystkie operacje mają się odbywać. Interfejs węzła **TimeSensor** jest następujący:

TimeSensor {

exposedField SFTIME **cycleInterval 1**

oznacza, że pełny cykl trwa 1 s.

exposedField SFBool **enabled TRUE**

exposedField SFBool **loop FALSE**

exposedField SFTIME **startTime 0**

exposedField SFTIME **stopTime 0**

eventOut SFTIME **cycleTime**

generuje zdarzenie (event) co 1 sekundę - czyli za każdym razem, gdy czujnik zakończy pełny cykl.

eventOut SFFloat **fraction_changed**

proporcja ukończenia bieżącego cyklu animacji (od 0 do 1).

eventOut SFBool **isActive**

eventOut SFTIME **time**

}

CZAS ANIMACJI - WĘZEŁ TIMESENSOR

- Pole **cycleInterval** typu **SFTime** definiuje szybkość animacji.
- Pole **enabled** typu **SFBool** odpowiada za to czy węzeł **TimeSensor** jest aktywny/nieaktywny.
- Pole **loop** typu **SFBool** odpowiada za to czy pętla animacji ma następować jedna po drugiej (wartość TRUE), czy też ma zostać wykonana tylko jedna (wartość FALSE).

Kiedy węzeł **TimeSesor** w polu **loop** ma wartość TRUE i wartość pola **startTime** jest **większa bądź równa** wartości w polu **stopTime** wtedy węzeł będzie wykonywał swoje zadanie w **nieskończoność** lub do momentu gdy wartość w polu **stopTime** będzie większa od wartości w polu **startTime**.

CZAS ANIMACJI - WĘZEŁ TIMESENSOR

Węzeł **TimeSensor** jest nieaktywny do momentu kiedy wartość pola **startTime** nie zostanie odczytana. Kiedy czas rzeczywisty równy jest wartości pola **startTime** zdarzenie **isActive** zostaje wygenerowane, przybiera wartość TRUE i węzeł się uaktywnia.

Wartości domyślne dla każdego pola węzła **TimeSensor** zostały tak ustawione, że jeśli nie podamy żadnych nowych wartości to po załadowaniu węzeł będzie nieaktywny i żadne ze zdarzeń nie zostanie wysłane. Węzeł **TimeSensor** możemy uaktywnić od razu po załadowaniu wpisując w pole **loop** wartość TRUE.

TRASA ANIMACJI

– WĘZŁY Z GRUPY INTERPOLATOR

Węzły z grupy **Interpolator** definiują trasę jaką dany obiekt będzie przebywał podczas animacji.

Węzły z grupy **Interpolator** definiują funkcję liniową $f(t)$ w przedziale od plus do minus nieskończoności. Funkcja definiowana jest przez n wartości t , które nazywamy **kluczami (key)** i n odpowiadających im wartości $f(t)$, które są nazywane **wartościami kluczy (keyValue)**. Ciąg kluczy musi być monotoniczny i niemalejący.

Węzeł interpolacji oblicza wartość funkcji $f(t)$ dla każdej wartości t , którą przyjmie zdarzenie **eventIn set_fraction**.

TRASA ANIMACJI

– WĘZŁY Z GRUPY INTERPOLATOR

```
OrientationInterpolator {  
    eventIn    SFFloat    set_fraction  
    exposedField MFFloat    key    []  
    exposedField MFRotation keyValue    []  
    eventOut    SFRotation value_changed  
}
```

Węzeł **OrientationInterpolator** interpoluje między wartościami typu **MFRotation** określającymi dane dotyczące obrotu obiektu. Obrót obiektu definiowany jest w polu **key** określając liczbę „momentów kluczowych animacji” czyli takich punktów, w których dany obiekt ma być obrócony o wartość określoną w polu **keyValue**. Węzeł korzystając z pól **key** i **keyValue** oblicza **funkcję interpolacyjną** obrotu obiektu. Liczba wartości w polu **key** i **keyValue** musi być identyczna.

Zdarzenia **set_fraction** i **value_changed** używane są do sterowania węzłem **OrientationInterpolator**.

TRASA ANIMACJI

– WĘZŁY Z GRUPY INTERPOLATOR

```
PositionInterpolator {  
    eventIn      SFFloat set_fraction  
    exposedField MFFloat key []  
    exposedField MFVec3f keyValue []  
    eventOut     SFVec3f value_changed  
}
```

Węzeł **PositionInterpolator** interpoluje między **punktami** zdefiniowanymi w przestrzeni trójwymiarowej. Liczba wartości w polu **key** i **keyValue** musi być identyczna. Zasady działania pól **key**, **keyValue** są analogiczne jak w węźle **OrientationInterpolator**.

TRASA ANIMACJI

– WĘZŁY Z GRUPY INTERPOLATOR

```
ColorInterpolator {  
    eventIn    SFFloat set_fraction  
    exposedField MFFloat key []  
    exposedField MFColor keyValue []  
    eventOut    SFColor value_changed  
}
```

Węzeł **ColorInterpolator** interpoluje między wartościami typu **MFColor**. Ilość definicji kolorów w polu **keyValue** powinna odpowiadać ilości wartości w polu **key**.

Wszystkie węzły interpolacji uruchamiamy wraz z przyjęciem zdarzenia **set_fraction**.

TRASA ANIMACJI

– WĘZŁY Z GRUPY INTERPOLATOR

```
CoordinateInterpolator {  
    eventIn      SFFloat set_fraction  
    exposedField MFFloat key []  
    exposedField MFVec3f keyValue []  
    eventOut     MFVec3f value_changed  
}
```

```
key      [ 0.0, 0.5, 1.0]  
keyValue [ 0 0 0, 10 10 30,  
          10 20 10, 40 50 50,  
          30 50 60, 40 50 60 ]
```

```
key = 3, keyValue = 6  
6 / 3 = 2
```

Węzeł **CoordinateInterpolator** interpoluje między **wektorami** typu **MFVec3f**. Każdej wartości w polu **key** musi odpowiadać **x** definicji wektorów w polu **keyValue**. **x** musi być taką liczbą, której wynik dzielenia z ilością wartości w polu **key** da liczbę całkowitą.

TRASA ANIMACJI

– WĘZŁY Z GRUPY INTERPOLATOR

```
NormalInterpolator {  
    eventIn      SFFloat set_fraction  
    exposedField MFFloat key []  
    exposedField MFVec3f keyValue []  
    eventOut     MFVec3f value_changed  
}
```

Węzeł **NormalInterpolator** interpoluje między prostopadłymi do określonej powierzchni **wektorami** typu **MFVec3f**. Każdej wartości w polu **key** musi odpowiadać **x** definicji wektorów w polu **keyValue**. **x** musi być taką liczbą, której wynik dzielenia z ilością wartości w polu **key** da liczbę całkowitą. **Można używać tego węzła do tworzenia ruchomych efektów cieniowania.**

POWIĄZANIE ZDARZEŃ – POLECENIE ROUTE

Połączenie między węzłem **generującym zdarzenie** a węzłem **przyjmującym zdarzenie** nazywane jest **trasą (route)**. Węzeł, który wysyła zdarzenia danego typu może być połączone trasą z węzłem przyjmującym zdarzenia tego samego typu.

```
ROUTE [nazwa_węzła].[zdarzenie_changed]  
      TO [nazwa_węzła].[set_zdarzenie]
```

- Trasy **nie są węzłami** tylko konstrukcją służącą do połączenia węzłów, między którymi przebiega zdarzenie.
- Komenda **ROUTE** może zostać zadeklarowana w dowolnym miejscu pliku *.x3dv.
- Wymagana jest zgodność typów pól.
- Trasy ustala się tylko od **eventOut** do **eventIn**. Jeżeli oba pola lub zdarzenia są rodzaju **exposedField** kolejność pola/zdarzenia, od którego zaczynamy nie jest istotna.

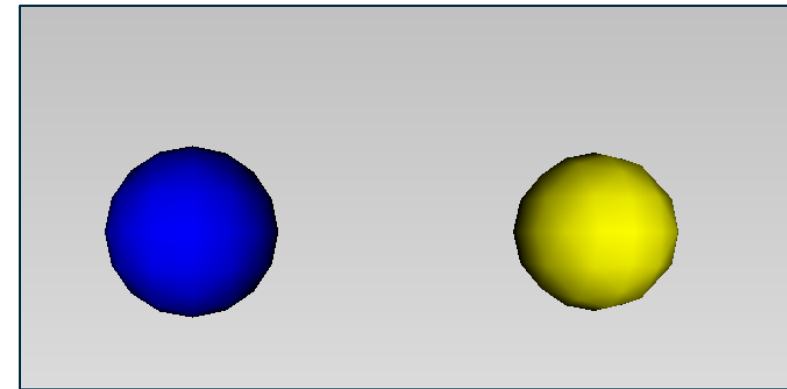
PIERWSZA ANIMACJA

ETAP 1: DEFINICJA OBIEKTÓW

#X3D V3.3 utf8

```
DEF Pierwszy Transform {  
  translation 0 0 0  
  children [  
    Shape {  
      appearance Appearance {  
        material Material {  
          diffuseColor 0 0 1  
        }  
      }  
      geometry Sphere {}  
    }  
  ]  
}
```

```
DEF Drugi Transform {  
  translation 5 0 0  
  children [  
    Shape {  
      appearance Appearance {  
        material Material {  
          diffuseColor 1 1 0  
        }  
      }  
      geometry Sphere {}  
    }  
  ]  
}
```



PIERWSZA ANIMACJA

ETAP 2: DEFINICJA WĘZŁÓW TIMESENSOR & POSITIONINTERPOLATOR

```
DEF Czas TimeSensor {  
    loop TRUE  
    cycleInterval 10  
}
```

```
DEF Inter PositionInterpolator {  
    key    [ 0, 0.25, 0.5, 0.75, 1 ]  
    keyValue [ 5 0 0, 0 5 0, -5 0 0, 0 -5 0, 5 0 0 ]  
}
```


PIERWSZA ANIMACJA

ETAP 3: DEFINICJA TRASY ROUTE

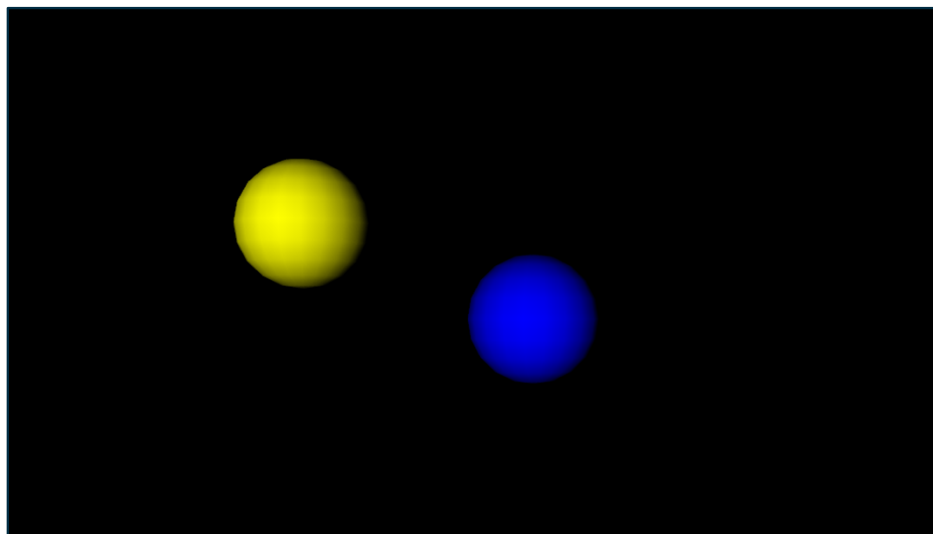
1. Węzeł TimeSensor jest domyślnie aktywowany.
2. Aktywacja węzła **PositionInterpolator** nastąpi poprzez przyjęcie zdarzenia **set_fraction**.

ROUTE Czas.fraction_changed TO Inter.set_fraction

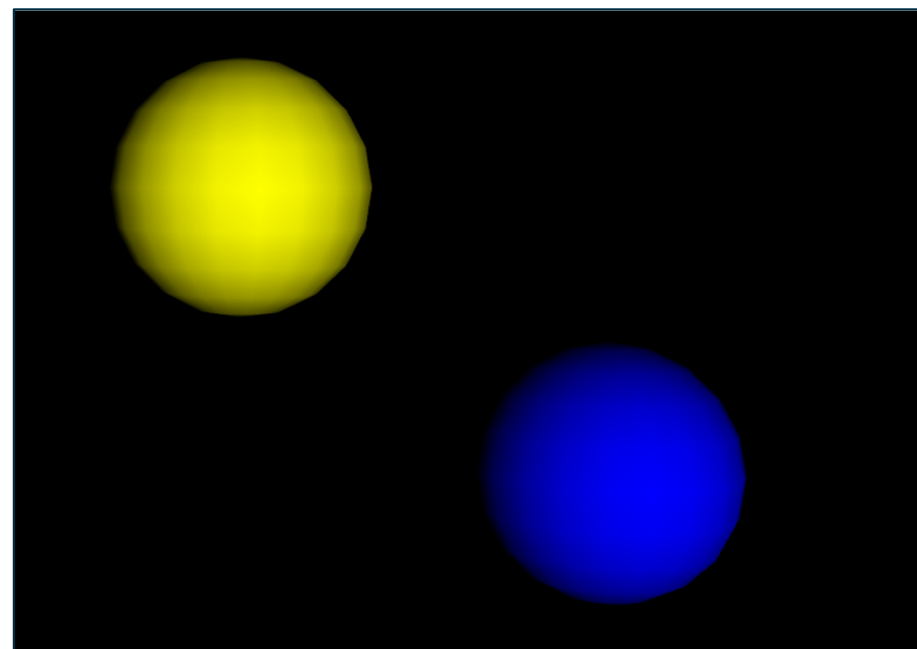
3. Przypisanie przesunięcie. Łączymy zdarzenie **value_changed** węzła **PositionInterpolator** ze zdarzeniem **set_translation** węzła **Transform**, w którym zdefiniowany jest nasz obiekt.

ROUTE Inter.value_changed TO Drugi.set_translation

PIERWSZA ANIMACJA



FOLLOW ME (viewpoint żółta kula)



ĆWICZENIA

UWAGI DO ĆWICZEŃ:

1. Zadania wykonujemy samodzielnie ☺.
2. Każde zadanie modelowe zapisujemy do osobnego pliku nadając mu nazwę odpowiadającą numerowi zadania.
3. Oficjalne specyfikacje dla standardu X3D (kodowanie klasyczne):

<https://www.web3d.org/documents/specifications/14772/V2.0/>

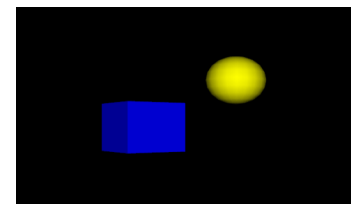
Pliki z swoimi rozwiązaniami umieszczamy w swoich indywidualnych katalogach na MS Teams.

ĆWICZENIA

01

Utwórz dwa obiekty: kulę oraz sześcian.

Zaprojektuj animację: sześcian obraca się wokół własnej osi a kula porusza się dookoła sześcianu po dowolnej ścieżce.



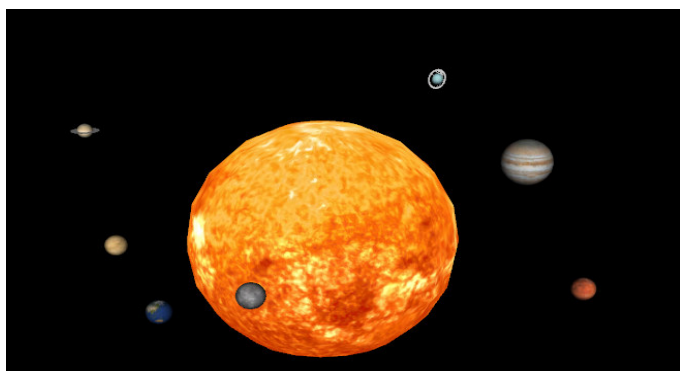
02

Zaprojektuj grę o motywie: **LABIRYNT**. Wykorzystaj poznane do tej pory węzły (Lab 1 i Lab 2) i polecenia:

- ściany labiryntu + podłoga,
- oświetlenie,
- otwarcie wejścia do labiryntu (animacja otwieranych drzwi).

03

Zaprojektuj układ słoneczny. Uwzględnij w modelu poznane węzły (m.in. tekstury, światło, punkty patrzenia na scenę 3D).



WAŻNE: Kod źródłowy labiryntu będzie wykorzystywany (uaktualniany) na kolejnych zajęciach 😊.

**DZIĘKUJĘ
ZA UWAGĘ**

LITERATURA

1. <https://www.web3d.org>
2. <https://www.x3dom.org>
3. <https://x3dgraphics.com>
4. K. Dąbkowski, VRML 97 trzeci wymiar sieci, Wydawnictwo MIKOM, Warszawa, 1998
5. S. Bucaro, X3Dom Primer: Learn the basics of how to code 3D objects and scenes to display on the web, bucarotechelp.com , 2018
6. D. Brutzman , L. Daly, X3D: Extensible 3D Graphics for Web Authors, Morgan Kaufmann, 2017